

实验 2：命题逻辑与 SAT 求解

1. 实验主题

把软件工程约束转换为命题逻辑和 CNF，使用 MiniSAT 求解，并检查编码、模型和解空间是否符合原始需求。

2. 实验目标

- 使用 MiniSAT 求解布尔可满足性问题（Boolean satisfiability problem, SAT），区分可满足（SAT）与不可满足（UNSAT）结果。
- 把自然语言约束编码为合取范式（conjunctive normal form, CNF）和 DIMACS CNF 文件格式，并维护变量—语义映射。
- 从 MiniSAT 模型恢复领域解，并独立检查每条原始约束。
- 使用阻塞子句（blocking clause，即排除当前模型、迫使求解器寻找不同模型的子句）分析无解、多解与唯一解。
- 使用 C++ 函数检查具体赋值是否满足 CNF，并通过完整比较有限变量上的模型集合判断两份编码是否等价。

允许使用 LLM 协助候选编码和脚本编写，但所有模型、反例和等价性结论都必须通过 MiniSAT 实际运行或完整赋值检查独立核验。若使用了 LLM，应在实验报告中简要说明其用途。

3. 工具与参考资料

- [MiniSAT 官方代码库](#)
- [SAT Competition: DIMACS CNF 说明](#)
- 可选：任意脚本语言，用于生成 CNF、解析模型和枚举多个解。

4. 最小环境检查（Smoke Test）

4.1 MiniSAT

保存以下内容为 `minisat_smoke.cnf`：

```
c x1 must be true
p cnf 1 1
1 0
```

运行：

```
minisat minisat_smoke.cnf smoke_result.txt
```

环境正确时，终端应报告 `SATISFIABLE`，结果文件应包含变量 1 为真的模型。随后把唯一子句改为 `-1 0` 再运行，也应得到 SAT，但模型中变量 1 为假。

5. 实验任务

任务 1：DIMACS 读写与结果解释

任务摘要： 读懂一个软件运行模式配置问题的 DIMACS 编码，通过修改约束构造不同版本，并从模型或约束传播过程解释求解结果。

某软件可以采用服务器模式或本地模式，并根据运行模式选择数据库或文件存储后端。定义变量如下：

变量	编号	为真时的含义
S	1	启用服务器模式
L	2	启用本地模式
D	3	启用数据库后端
F	4	启用文件存储后端
P	5	启用持久化功能

配置规则如下：

- 1. 服务器模式和本地模式至少启用一种。
- 2. 服务器模式要求数据库后端。
- 3. 本地模式要求文件存储后端。
- 4. 数据库后端要求启用持久化功能。
- 5. 文件存储后端要求启用持久化功能。
- 6. 启用持久化功能时，至少启用一种存储后端。
- 7. 数据库后端与文件存储后端不能同时启用。
- 8. 当前产品版本要求启用持久化功能。

保存以下内容为 `base.cnf`：

```
c S=1, L=2, D=3, F=4, P=5
p cnf 5 8
1 2 0
-1 3 0
-2 4 0
-3 5 0
-4 5 0
-5 3 4 0
-3 -4 0
5 0
```

- 1. 把八条业务规则分别写成命题逻辑公式，并建立“规则编号—逻辑公式—DIMACS 子句”对照表；同时检查 `base.cnf` 的文件头、变量编号、子句数量和结尾的 `0`。
- 2. 运行 `base.cnf`，解读 MiniSAT 的终端输出和结果文件：说明求解状态表示什么，解释模型中正、负变量编号的含义，把模型还原为软件配置，并逐条检查该配置是否满足八条业务规则。
- 3. 分别构造并运行两个变体：在基础文件中增加单元子句 `1 0` 得到 `server_only.cnf`，表示必须启用服务器模式；把基础文件最后的 `5 0` 改为 `-5 0` 得到 `no_persistence.cnf`，表示禁止持久化。正确更新文件头，并比较三个版本的求解状态和模型变化。

4. 对 `no_persistence.cnf`，按照“持久化—存储后端—运行模式”的顺序解释为什么约束最终发生冲突，并列出这条冲突链实际使用的业务规则。

实验报告应包含： 八条规则的逻辑公式与 DIMACS 对照、三个 DIMACS 文件、MiniSAT 求解状态和模型输出的逐项解读、SAT 模型的业务解释，以及 `no_persistence.cnf` 的冲突链；能够看清三个版本求解状态和模型的关键截图；以及你对“求解结果为什么必须结合变量语义和约束关系解释”自行撰写的结论。

任务 2：诚实者与说谎者建模

任务摘要： 把人物陈述转换为命题逻辑，判断问题是无解、唯一解还是多解，并解释判断依据。

场景如下：Alice 说“Bob 与 David 身份不同”；Bob 说“Alice 或 David 中至少一人诚实”；Carol 说“Alice 是说谎者”；David 说“Alice 或 Bob 中至少一人诚实”；Eve 说“Carol 是诚实者”。另有“至少两名诚实者、至少两名说谎者”。

1. 明确区分“某人诚实”和“某人的陈述为真”，用等价关系表达身份规则，再转换为 CNF。
2. 运行 MiniSAT。如果找到一个模型，加入排除该模型的阻塞子句并再次求解，根据两次求解结果判定原问题是唯一解还是多解；如果第一次就是 UNSAT，则判定原问题无解，并从人物陈述的逻辑关系中解释一组直接冲突。

实验报告应包含： 变量表、人物陈述的逻辑公式、CNF 转换、第一次与第二次求解结果，以及无解、唯一解或多解的判断依据；能够看清求解状态和模型的关键截图；以及你对该问题解的数量自行撰写的结论。

任务 3：自选场景的 SAT 建模

任务摘要： 使用 LLM 为自选的软件工程场景生成 SAT 模型，整理自然语言需求与 CNF 子句的对应关系，再使用 MiniSAT 求解。

从软件配置、测试用例选择、课程冲突或资源分配中选择一个场景。最终模型应包含不少于 10 个有业务含义的布尔变量和 15 个 CNF 子句，并至少包含依赖、互斥、至少一个、至多一个和条件组合中的三类关系。最终需求应当至少允许一个合法方案。

1. 编写提示词，让 LLM 生成场景说明、变量含义、编号后的自然语言需求、对应的 CNF 子句，以及一份满足全部需求的候选方案。提示词中应写明上述规模和可满足性要求。保存你实际提供给 LLM 的完整提示词。
2. 整理并检查 LLM 生成的模型，修正不合理或不一致的内容，形成最终的“自然语言需求—CNF 子句”对照，并把候选方案逐条代入最终需求和 CNF 检查。
3. 将最终 CNF 写成 DIMACS 文件并使用 MiniSAT 求解，把求得的模型还原为业务方案并逐条检查需求。若 MiniSAT 报告 UNSAT，应检查需求或编码并修正，直到最终提交的模型可满足。

实验报告应包含： 提供给 LLM 的完整提示词、最终变量表和“自然语言需求—CNF 子句”对照、LLM 给出的候选方案及你的逐条检查、DIMACS 文件、MiniSAT 求得的业务方案及需求检查；关键求解结果的截图；以及你对最终 CNF 是否准确表达需求自行撰写的结论。

任务 4（编程题）：实现 CNF 赋值检查器

任务摘要： 按照固定接口实现一个 C++ 函数，判断给定布尔赋值是否满足一组 CNF 子句，并观察一次单变量翻转怎样使具体子句失效。

请在 `lab2_solution.cpp` 中实现以下函数：

```
#include <vector>

bool satisfiesCnf(
    int variableCount,
    const std::vector<std::vector<int>>& clauses,
    const std::vector<int>& assignment) {
    // 在此处完成实现
}
```

接口约定：

- 变量编号为 1 到 `variableCount`。
- `assignment` 恰好包含 `variableCount` 个元素；`assignment[i]` 表示变量 `i + 1` 的值，只可能为 0 或 1。
- 正整数文字表示对应变量为真，负整数文字表示对应变量为假。
- 每个子句至少包含一个文字，且文字的绝对值不超过 `variableCount`。
- 只有每个子句都至少有一个文字为真时，函数才返回 `true`。
- 函数不得调用 MiniSAT，不得定义 `main`，不得进行标准输入输出；评分程序将使用 C++17 编译并直接调用该函数。

完成以下工作：

1. 实现 `satisfiesCnf`，正确处理正文字、负文字、单文字子句和多个子句。
2. 自行设计至少六个测试，其中应包含满足赋值和不满足赋值。
3. 使用固定 CNF `{{1, 2}, {-1, 3}, {-2, -3}}` 和赋值 `x1=true, x2=false, x3=true` 检查函数，确认其返回 `true`。随后只把 `x3` 改为 `false`，再次运行函数，并指出哪个子句因此变为不满足。

实验报告应包含： 最终提交代码、六个测试、固定赋值及翻转 `x3` 前后的函数结果、翻转后不满足的子句；批量测试和固定用例检查的关键截图；以及你对 CNF 中“所有子句均满足”含义自行撰写的结论。

任务 5：使用阻塞子句寻找多个解

任务摘要： 每找到一个解，就增加一条排除该解的阻塞子句，继续求解，从而枚举同一组业务规则下的多个不同方案。

阻塞子句（blocking clause）用于排除求解器刚刚找到的完整赋值。例如当前模型为 `x1=true, x2=false, x3=true`，可加入 `(-x1 OR x2 OR -x3)`；这条子句只排除该模型，不直接指定下一个答案。

本任务使用一个固定的测试策略选择问题，不依赖任务 3 的自选模型。定义变量如下：

变量	编号	为真时的含义
U	1	执行单元测试
I	2	执行集成测试
S	3	执行静态分析
C	4	收集覆盖率

规则如下：单元测试和集成测试至少执行一种；静态分析要求执行单元测试；覆盖率收集要求执行集成测试；静态分析和覆盖率收集不能同时启用。保存为 `test_strategy.cnf`：

```
c U=1, I=2, S=3, C=4
p cnf 4 4
1 2 0
-3 1 0
-4 2 0
-3 -4 0
```

1. 运行原始 CNF，把第一个模型还原为测试策略并逐条检查四条规则。
2. 根据每次得到的完整模型构造阻塞子句，重复运行 MiniSAT，直到结果变为 UNSAT。列出全部不同测试策略和每次加入的阻塞子句。
3. 每得到一个模型，都使用任务 4 的 `satisfiesCnf` 检查原始四条子句，并确认新模型在四个业务变量上与此前模型不同。根据最终的 UNSAT 说明为什么已经没有遗漏其它完整赋值。
4. 增加业务规则“必须执行静态分析”，即加入单元子句 `3 0`，重新完整枚举。比较修改前后的方案数量，并解释哪些方案被新规则排除。

实验报告应包含： 原始 CNF 的全部业务方案、连续加入的阻塞子句、每个方案的赋值检查结果、最终 UNSAT，以及加入 `3 0` 前后的方案数量和被排除方案；连续求解过程的关键截图；以及你对阻塞子句为什么能够逐个排除已有完整赋值自行撰写的结论。

任务 6：检查两份 CNF 是否等价

任务摘要： 系统检查两份写法不同的候选 CNF 是否具有相同模型集合，解释它们为何等价，再通过删除一个子句制造差异并寻找区分赋值。

某服务包含以下六个业务变量：

变量	编号	为真时的含义
A	1	启用 API 服务
W	2	启用 Web 界面
D	3	启用数据库
C	4	启用缓存
L	5	启用日志
M	6	启用监控

需求如下：

1. API 服务和 Web 界面至少启用一个。
2. API 服务启用当且仅当数据库启用。
3. Web 界面启用时必须启用缓存。
4. 缓存启用时必须启用数据库。
5. 数据库启用时必须启用日志。
6. 监控启用当且仅当日志启用。

下面是两份候选编码。它们的子句数量和具体形式不同，不要仅根据文件外观判断二者是否等价。保存第一份为 `encoding_a.cnf`：

```
c A=1, W=2, D=3, C=4, L=5, M=6
p cnf 6 8
1 2 0
-1 3 0
-3 1 0
-2 4 0
-4 3 0
-3 5 0
-6 5 0
-5 6 0
```

保存第二份为 `encoding_b.cnf`：

```
c A=1, W=2, D=3, C=4, L=5, M=6
p cnf 6 12
-4 3 0
-3 5 0
-6 5 0
-5 6 0
1 2 4 0
1 2 -4 0
-1 3 6 0
-1 3 -6 0
-3 1 2 0
-3 1 -2 0
-2 4 5 0
-2 4 -5 0
```

两个 CNF 在业务变量上等价，是指对 `A`、`W`、`D`、`C`、`L`、`M` 的每一种完整赋值，两份 CNF 的满足结果都相同。能够满足其中一份而不满足另一份的赋值称为区分赋值（distinguishing assignment）。如果不存在区分赋值，两个 CNF 的模型集合相同。

1. 把六条需求写成命题逻辑公式，并分别运行两份编码，记录求解状态和一个模型。两份编码都是 SAT 只能说明它们各自至少有一个模型，不能据此判断模型集合是否相同；等价性由下一步的完整赋值检查确定。
2. 设计一个检查函数，枚举六个变量的全部 $2^6=64$ 种赋值，并调用任务 4 的 `satisfiesCnf` 比较 A 和 B 在每种赋值下的结果，从而判断两个编码是否等价。函数的接口和结果记录方式由你决定。使用该函数检查两份原始编码，并在实验报告中提交函数代码、检查方法和结果。
3. 使用命题逻辑等价变换，逐步推导为什么下面的公式成立，并说明这项变换为什么改变了 CNF 的写法和子句数量，却没有改变模型集合：

$$K \text{ 等价于 } (K \text{ OR } x) \text{ AND } (K \text{ OR NOT } x)$$

其中 K 可以是一个包含多个文字的子句， x 是另一个布尔变量。推导中应写明所使用的分配律、矛盾律和同一律。

4. 删除 B 中的子句 1 2 4 0 并把子句数改为 11。使用任意可靠方法找出一条使原始编码 A 与修改后的编码 B 得到不同结果的赋值。列出 A、W、D、C、L、M 的真假，将其翻译为完整的服务配置；再把该赋值分别代入两个编码，指出结果不同的具体子句和对应的业务需求。

实验报告应包含： 六条需求的逻辑公式、两份原始编码的 MiniSAT 结果、你设计的检查函数及其等价性检查结果、子句拆分公式的完整等价推导，以及修改 B 后找到的区分赋值、完整服务配置和两份编码的回代检查；能够看清原始模型集合比较和修改后区分赋值的关键截图；以及你对“语法形式不同为什么仍可能表达相同模型集合”自行撰写的结论。

6. 评分标准

本实验总分为 100 分。编程题根据自动测试结果和报告中的测试分析评分，其余任务根据模型、运行结果、截图和分析评分。

任务	分值
任务 1：DIMACS 读写与结果解释	15
任务 2：诚实者与说谎者建模	15
任务 3：自选场景的 SAT 建模	20
任务 4：实现 CNF 赋值检查器	20
任务 5：使用阻塞子句寻找多个解	15
任务 6：检查两份 CNF 是否等价	15
总分	100